

Software Engineering

CE2006/CZ2006

Requirements Analysis: Dynamic Models 1

Discussion Topics

- Interaction Diagrams
 - Sequence Diagram
 - Communication Diagram
- Reading
 - Bruegge chap. 2.4.3, 5.4.4
 - Fowler chap.4 and chap. 12 Bruegge chap.2.4.4, 5.4.9
 - Fowler chap.10

Requirement Analysis Goals

- Conceptual model (structural aspect)
 - Analyze use cases to identify the **objects** and **roles** of objects involved in the system
- Dynamic model (dynamic aspect)
 - Determine how to **fulfill the processes** defined in the use cases and which **objects** **do these processes**

Document and Visualize Dynamic Model

- Using the following types of diagrams
 - Sequence diagram
 - Communication diagram
 - State machine
 - Activity diagram

Sequence Diagram and Communication Diagram

Captures how **a group of objects** (in a Class Diagram) **interact or collaborate** to achieve an activity as described in a Use Case.

Usually captures the dynamics of **a single scenario** and the **sequence of messages** that are passed between the objects and the **sequence of events** that happen during the scenario.

Both diagrams essentially show the same information –

- **Sequence diagrams** emphasise the sequence and timing of calls and messages between the objects
- **Communication diagrams** emphasise the communication links between the objects

UML in Development Process

- Ultimate Goal: effective communication!
 - Requirement Elicitation (users & customers)
 - Design (designers & developers)
 - Documentation (other developers/maintainers)



How to Describe Systems' Interactions Using UML

Sequence Diagram

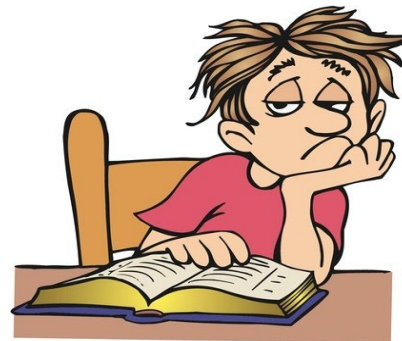
<https://youtu.be/Mqy569thFN8>

Example: EnterSecureFacility Use Case Description

Flow of Events:

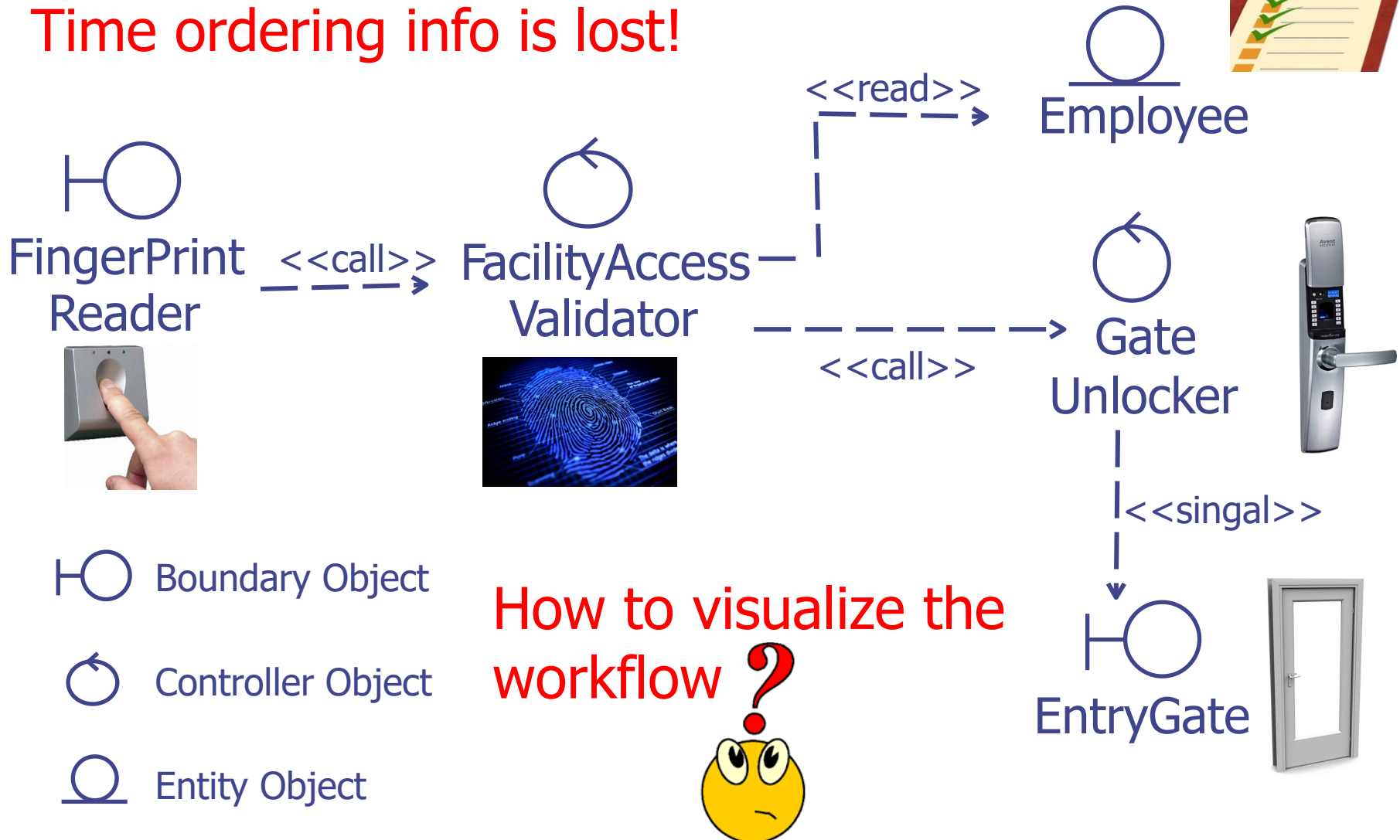
1. User presses on FingerprintReader.
2. FingerprintReader scans fingerprint and validates image with the FacilityAccessValidator.
3. The FacilityAccessValidator finds fingerprint in the EmployeeList.
4. The FacilityAccessValidator validates fingerprint and unlocks EntryGate.
5. ...

Too much text!



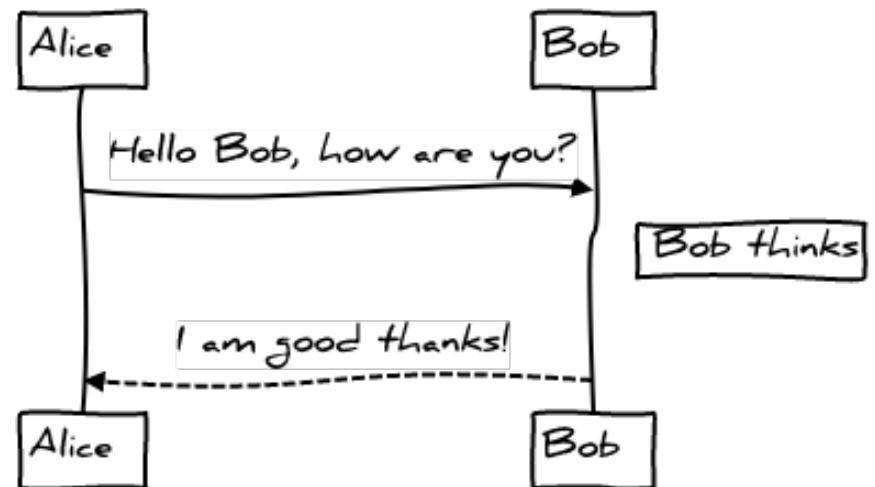
Class Diagram

Time ordering info is lost!

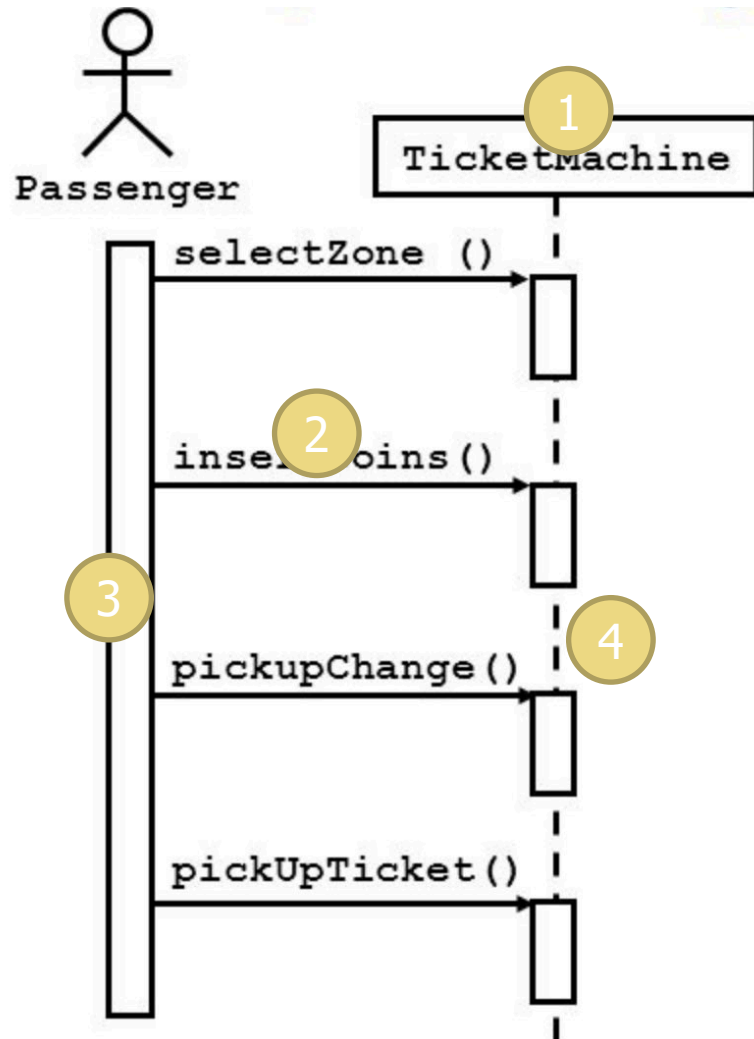


Sequence Diagram

- Captures how a group of objects interact or collaborate to achieve a use case scenario.
- Usually captures the dynamics of a **single scenario** and the **sequence of messages** that are passed between the objects.
- Emphasizes the **time ordering** of messages.



Sequence Diagram: Definitions



- Used during **requirement analysis**
 - To refine use case descriptions
 - Find additional/missing objects
- 1. **Objects** are represented by rectangles
- 2. **Messages** are represented by arrows
- 3. **Activations** are represented by narrow bars
- 4. **Lifelines** are represented by dashed lines

Example: EnterSecureFacility Use Case Description

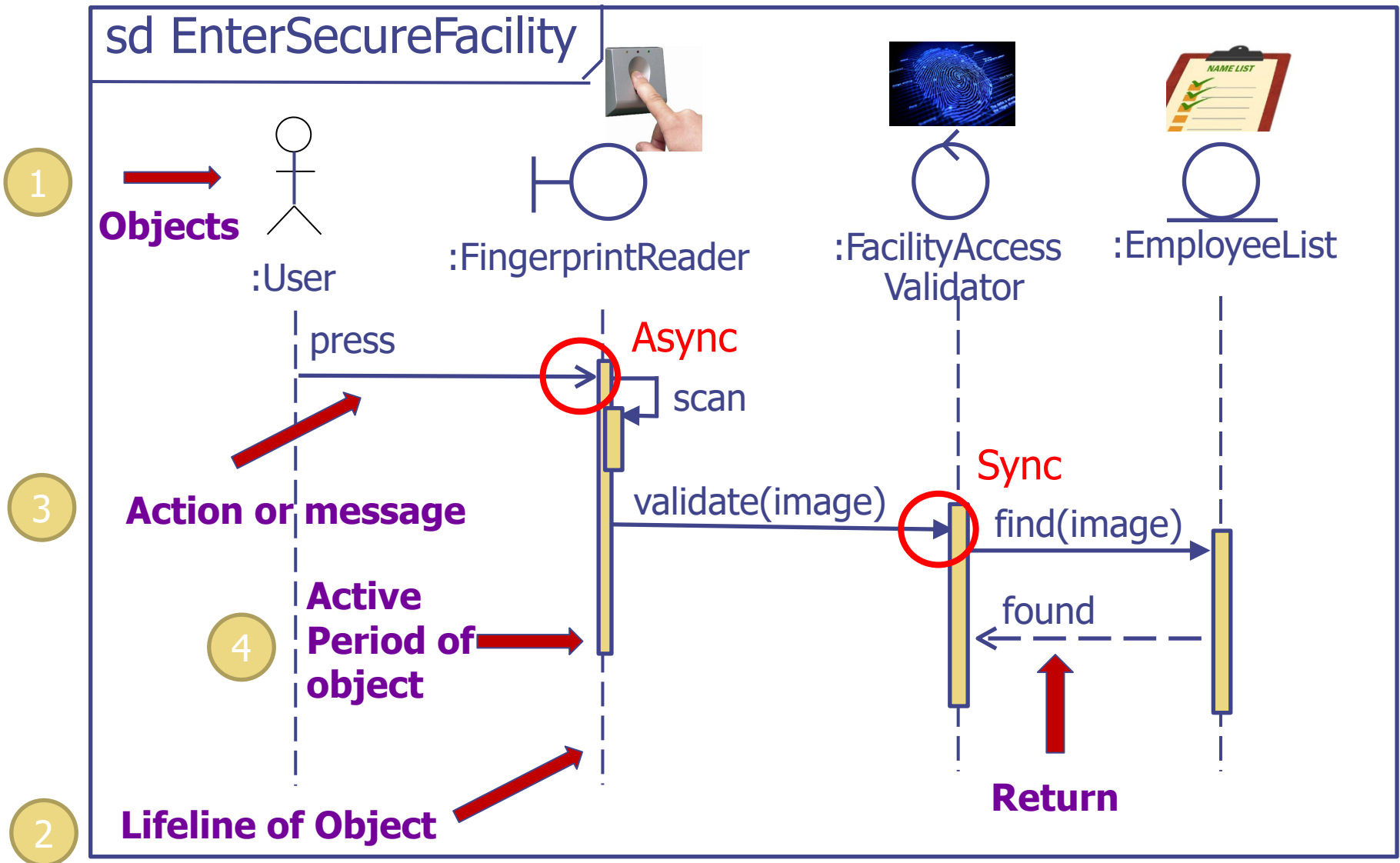
Flow of Events:

Drawing sequence diagram:
Identify important verbs

1. User presses on FingerprintReader.
2. FingerprintReader scans fingerprint and validates image with the FacilityAccessValidator.
3. The FacilityAccessValidator finds fingerprint in the EmployeeList.
4. The FacilityAccessValidator validates fingerprint and unlocks EntryGate.
5. ...



Example: EnterSecureFacility Sequence Diagram



Refine EnterSecureFacility Use Case Description

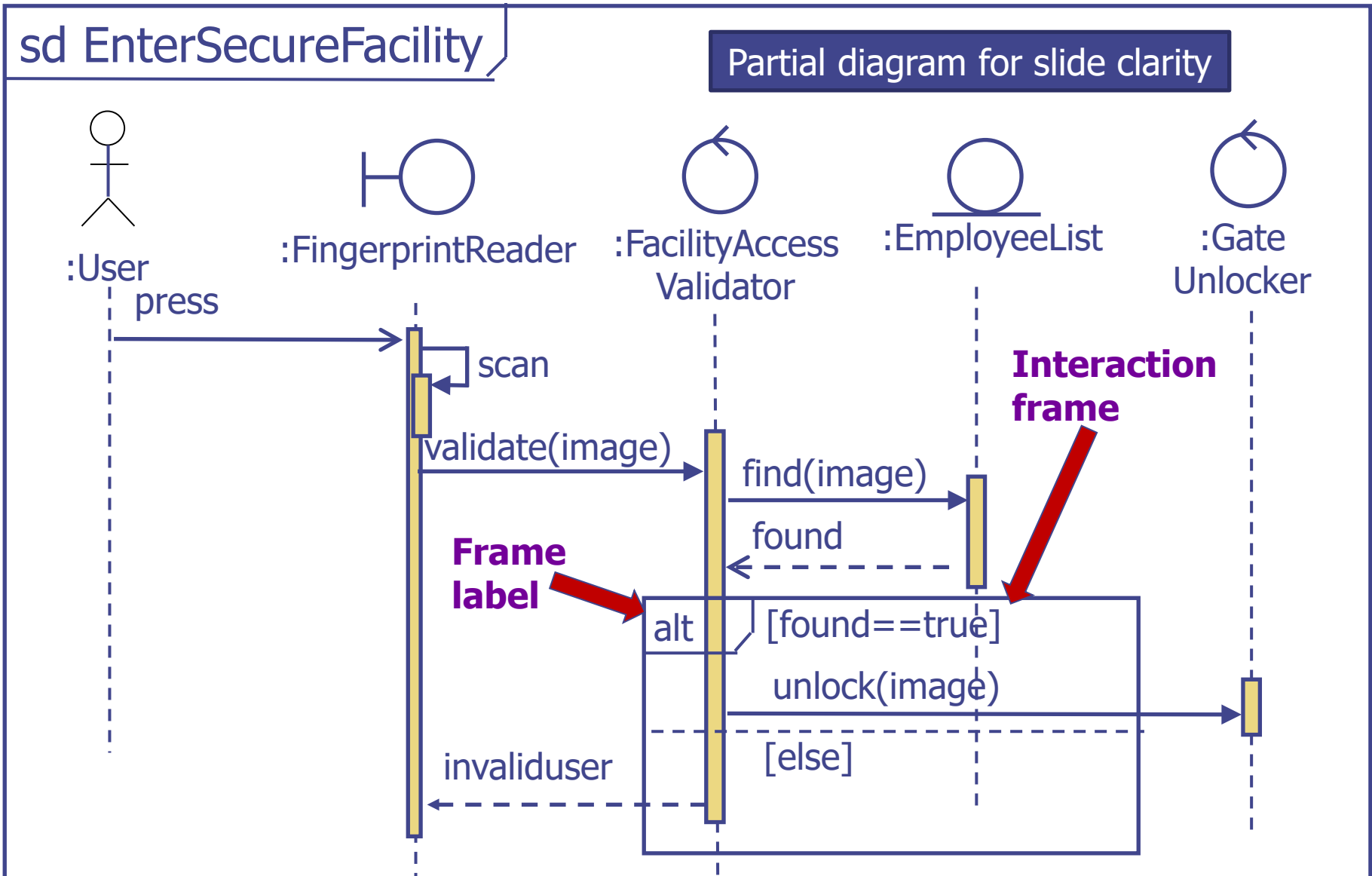
Flow of Events:

1. User presses on FingerprintReader.
2. FingerprintReader scans fingerprint and sends the image to FacilityAccessValidator.
3. The FacilityAccessValidator validates fingerprint with the scanned fingerprint image.
4. If the fingerprint image is found in the employee list, the FacilityAccessValidator unlocks EntryGate.
5. ...

Alternative Flows:

- AF-4. If the fingerprint image is not found in the employee list
1. FingerprintReader displays a "Please try again" message
 2. Go back to Step 1

Example: EnterSecureFacility Sequence Diagram



Types of Interaction Frames

sd sequence diagram

ref reference another sequence diagram

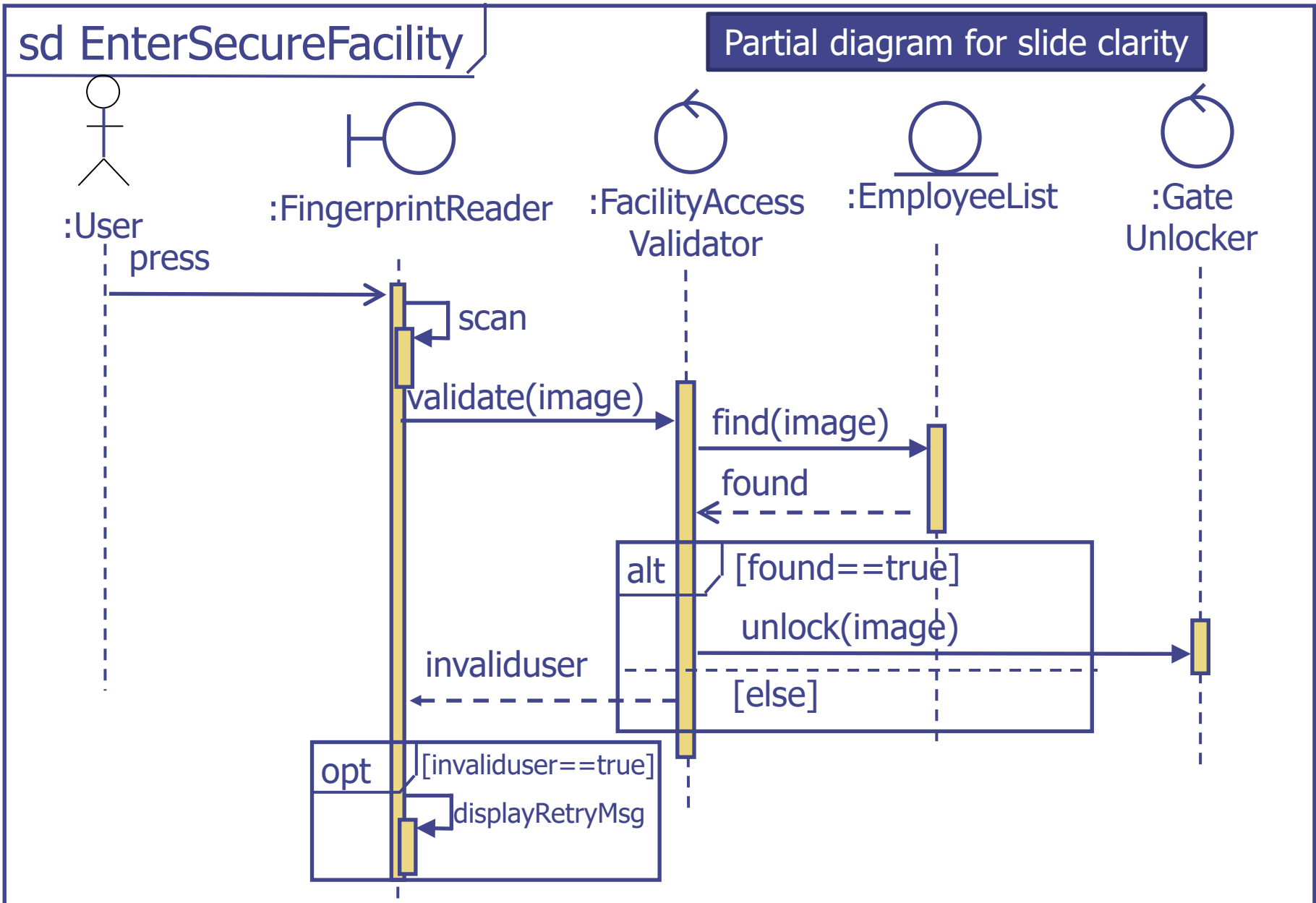
loop the actions loop until guard condition is false
(as in while, for)

opt/alt optional/alternative flow
(as in if-then, if-then-else, switch-case)

par each action is run in parallel

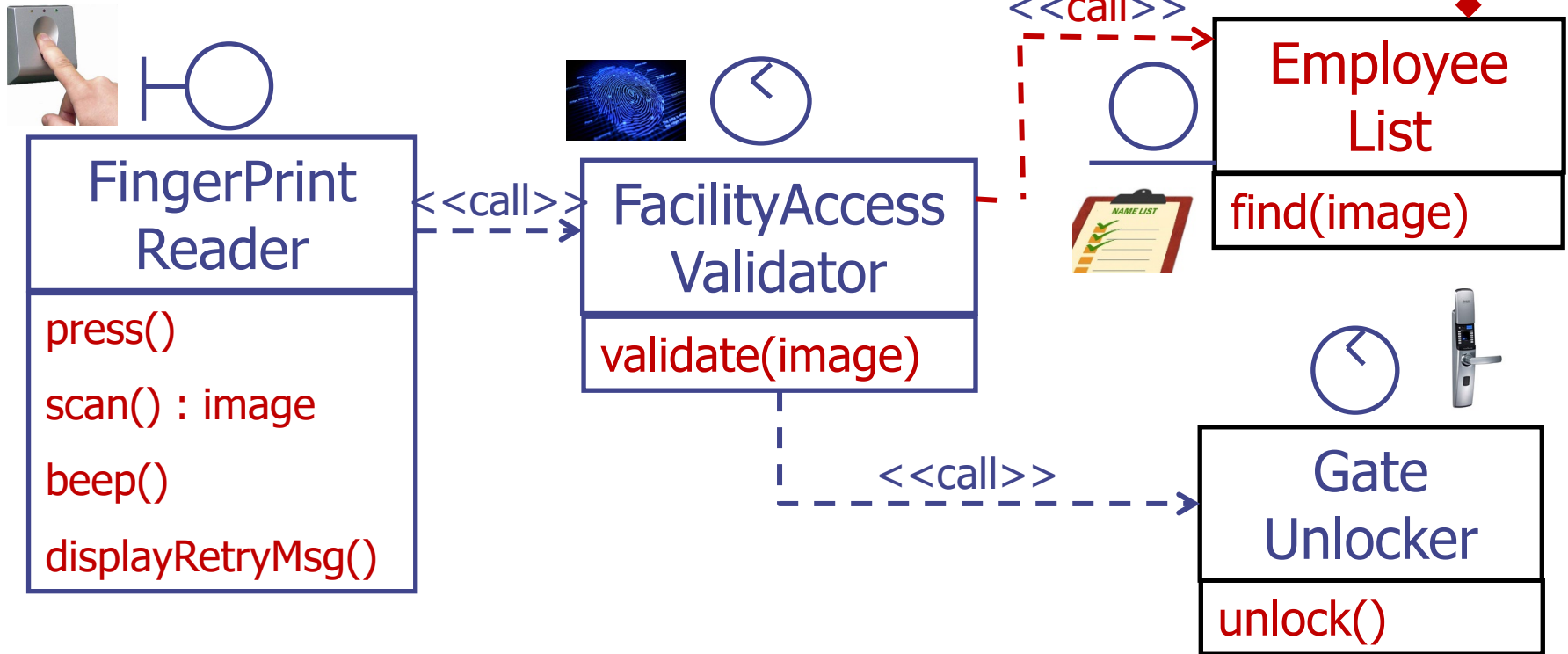
See UML cheatsheet for examples!

Refine EnterSecureFacility Sequence Diagram



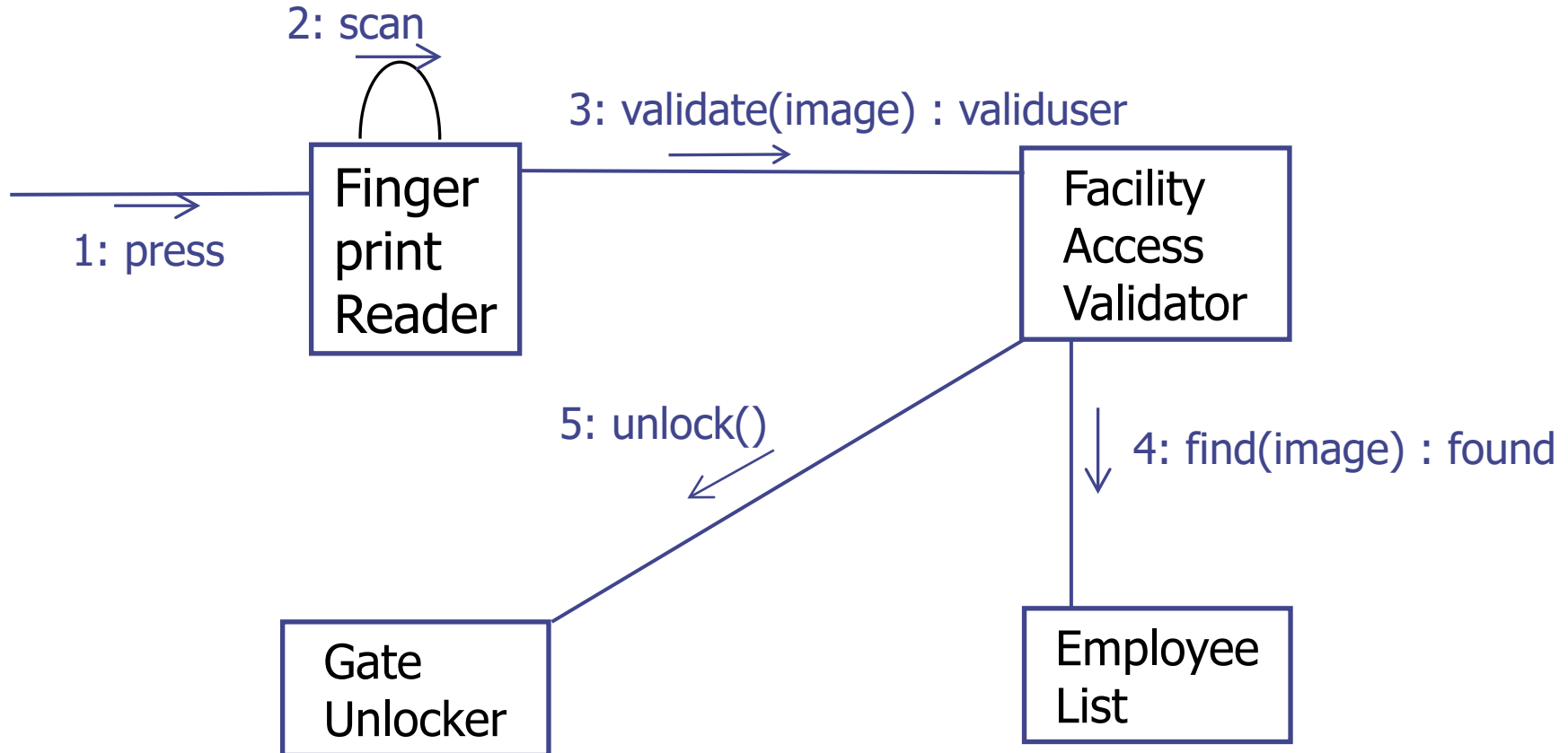
Refine Class Diagram

Partial diagram for slide clarity



Example: EnterSecureFacility Communication Diagram

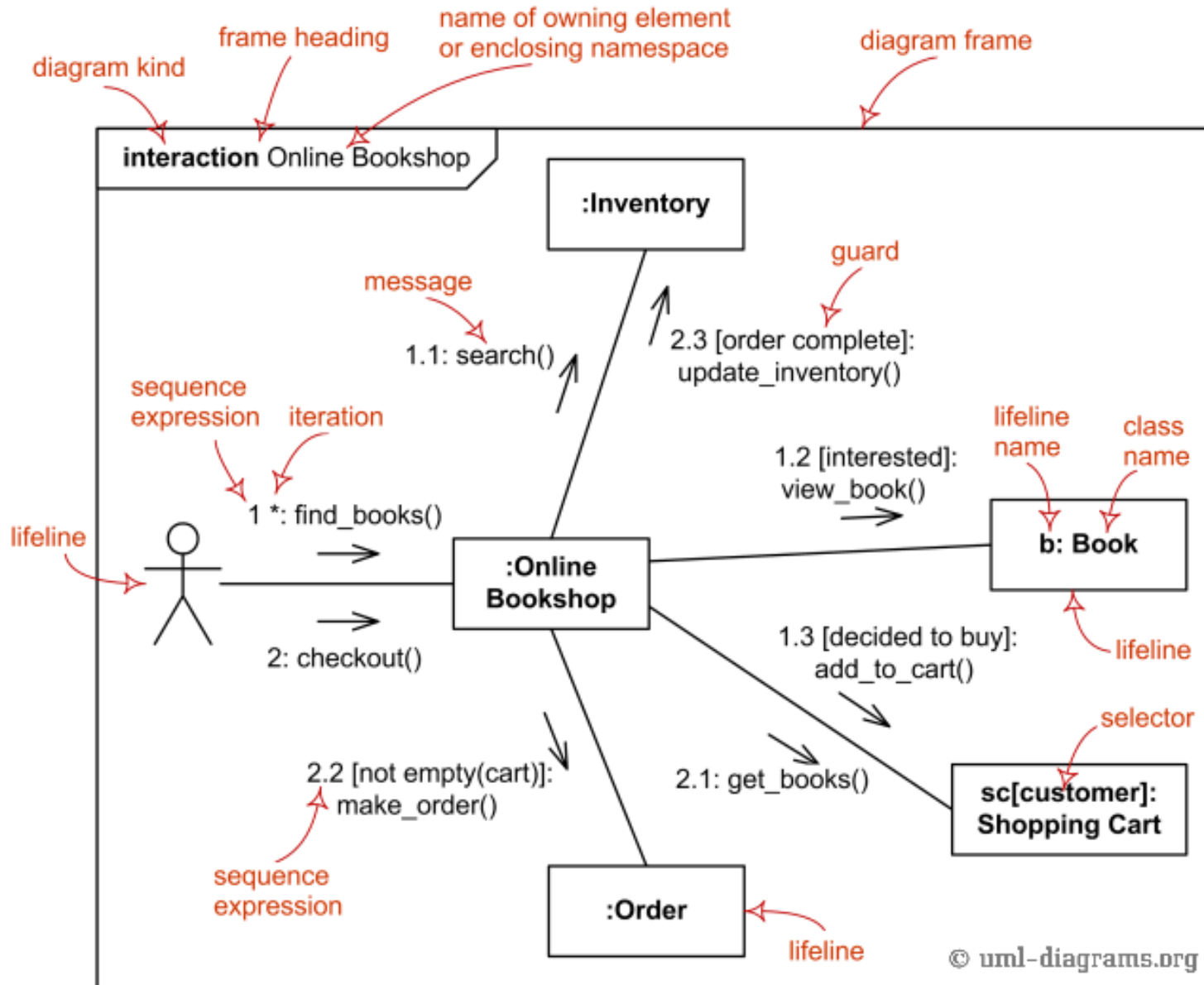
Partial diagram for slide clarity



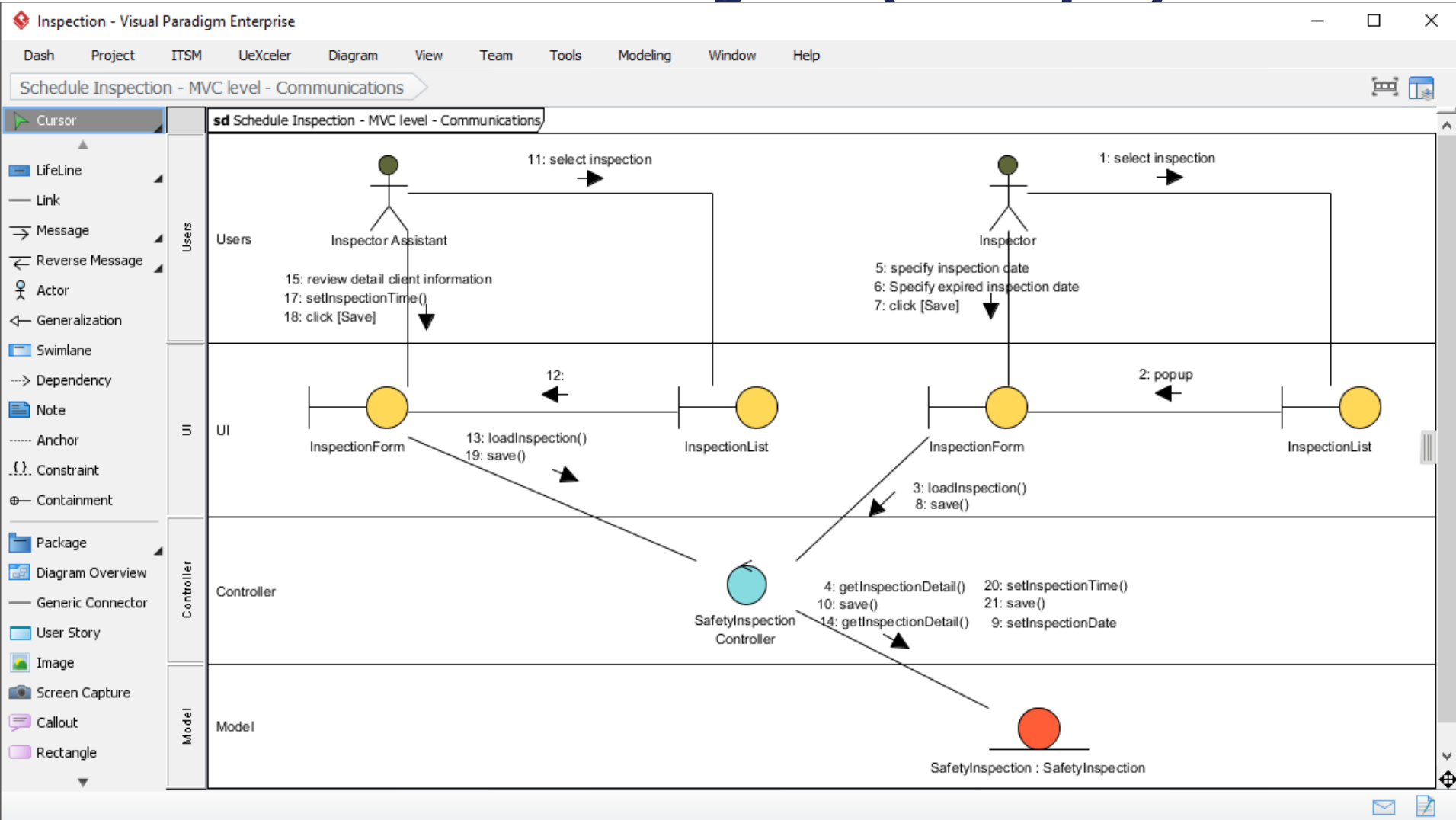
Communication diagram

- Models the interaction between objects or parts in terms of sequenced messages.
- Represents a combination of information taken from Class, Sequence, and Use-Case Diagrams describing both the static structure and dynamic behavior of a system.
- Messages are labeled with a chronological number.
- Compare with sequence diagram:
 - Communication diagrams show which elements each one interacts with better
 - Sequence diagrams show the order in which the interactions take place more clearly.

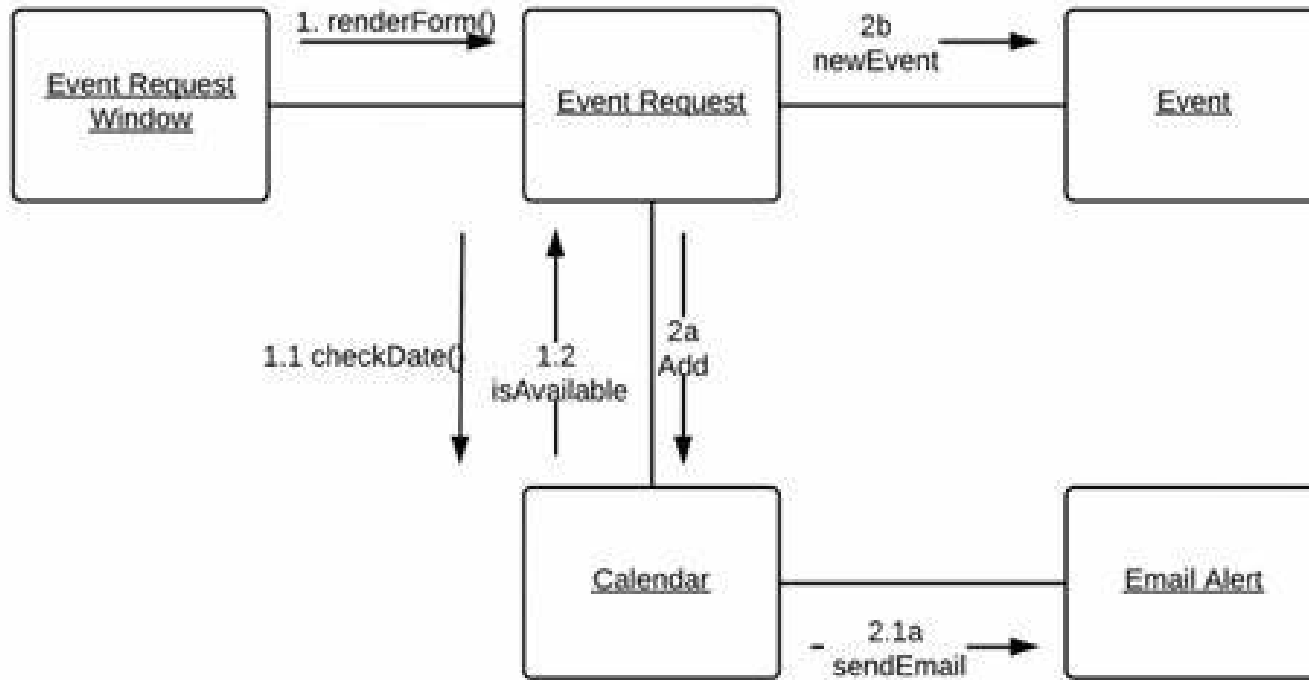
Communication diagram (example)



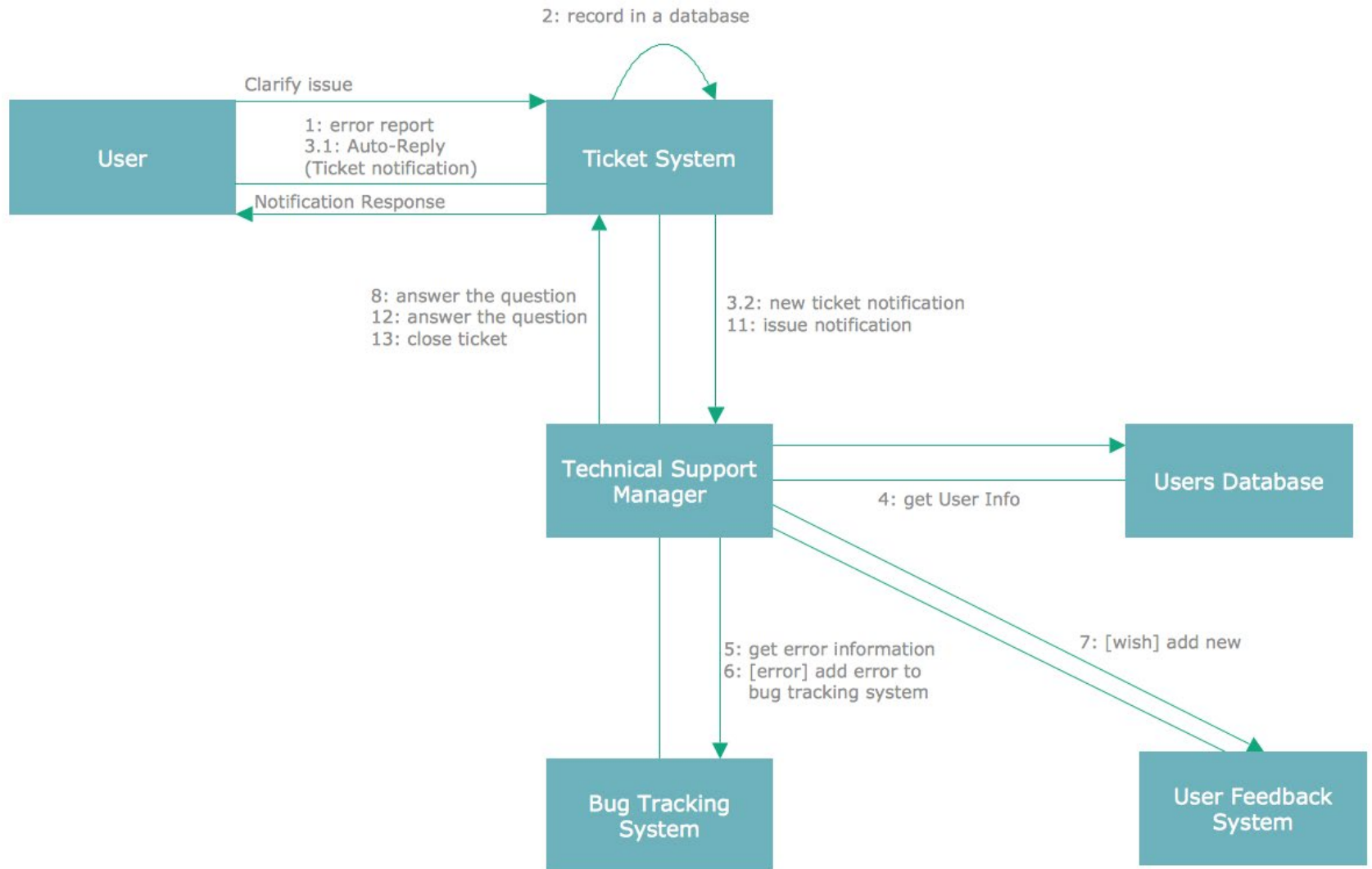
Communication diagram (example)



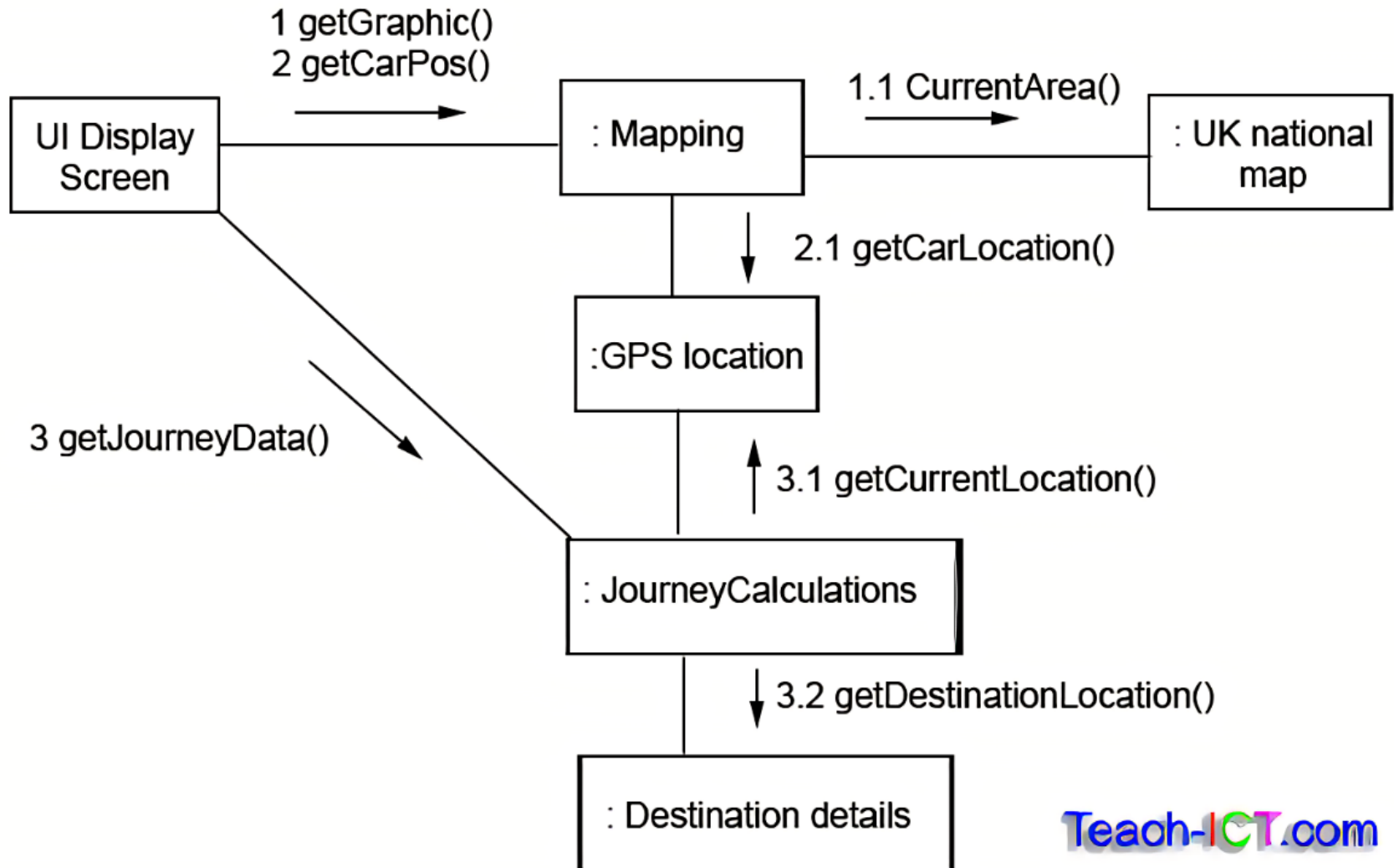
Communication diagram (example)



Communication diagram (example)



Communication diagram (example)



Summary

Sequence Diagrams are useful for:

Refining the Use Cases

- **Identifying missing steps and flows**
- **Identifying unnecessary steps and flows**

Refining the Class Diagrams (objects)

- **Identifying missing classes**
- **Identifying unnecessary classes**
- **Identifying class operations**
- **Identify class attributes**

Identifying all relationships between classes

Visual Paradigm can automatically switch between the two if you create the Sequence Diagram first.

Communication Diagrams serve a similar purpose by visualizing the sequence of messages differently.

In general, you need a sequence or communication diagram for each use case.

Summary

- Why sequence diagram?
 - Use cases are too wordy and also hard to read
 - Class diagram loses information on time ordering
 - Sequence diagram is more detailed than use case, not for non-technical communication in general.
- When to use sequence diagram?
 - Good at capturing interactions between objects
 - Use sequence diagram to look at the behaviour of **several objects within a single use case.**
 - Not so good at precise definition of behaviours
 - State diagrams (single object in many use cases).
 - Activity diagram (many use cases and many threads).
 - We will look at them later in the course.